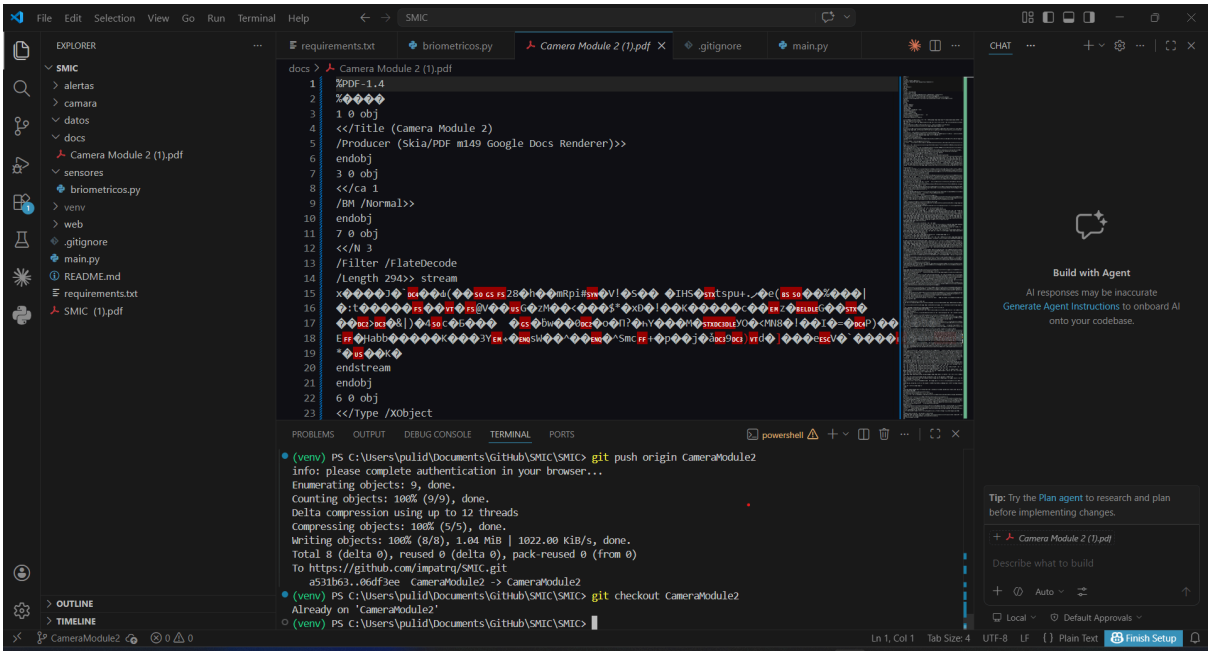
CAMERA MODULE 2

Paso a paso:

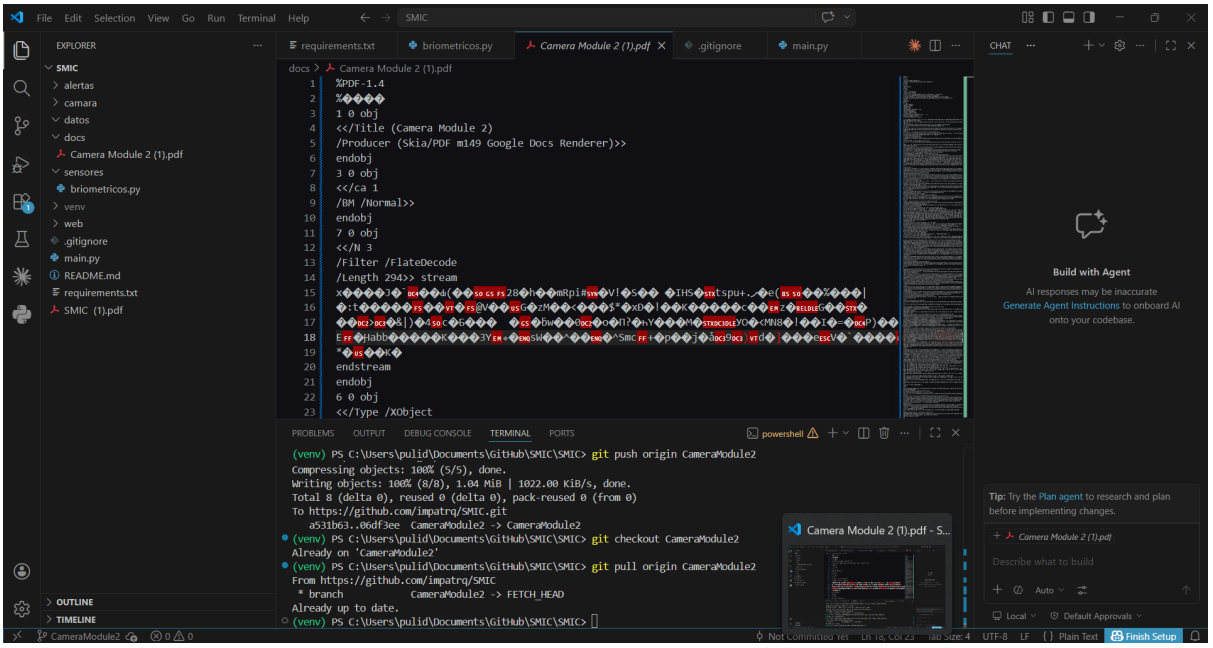
Una vez subido lo del dia anterior, aseguramos que estemos trabajando en la rama correcta

git checkout CameraModule2



Actualizamos la rama con los ultimos cambios hechos por el equipo

git pull origin CameraModule2



Y activamos el entorno virtual

venv\Scripts\activate

Eliminamos la carpeta sensores ya que no la utilizaremos y actualizamos el repo en github

Primero verificamos los cambios

git status

Luego agregamos los cambios

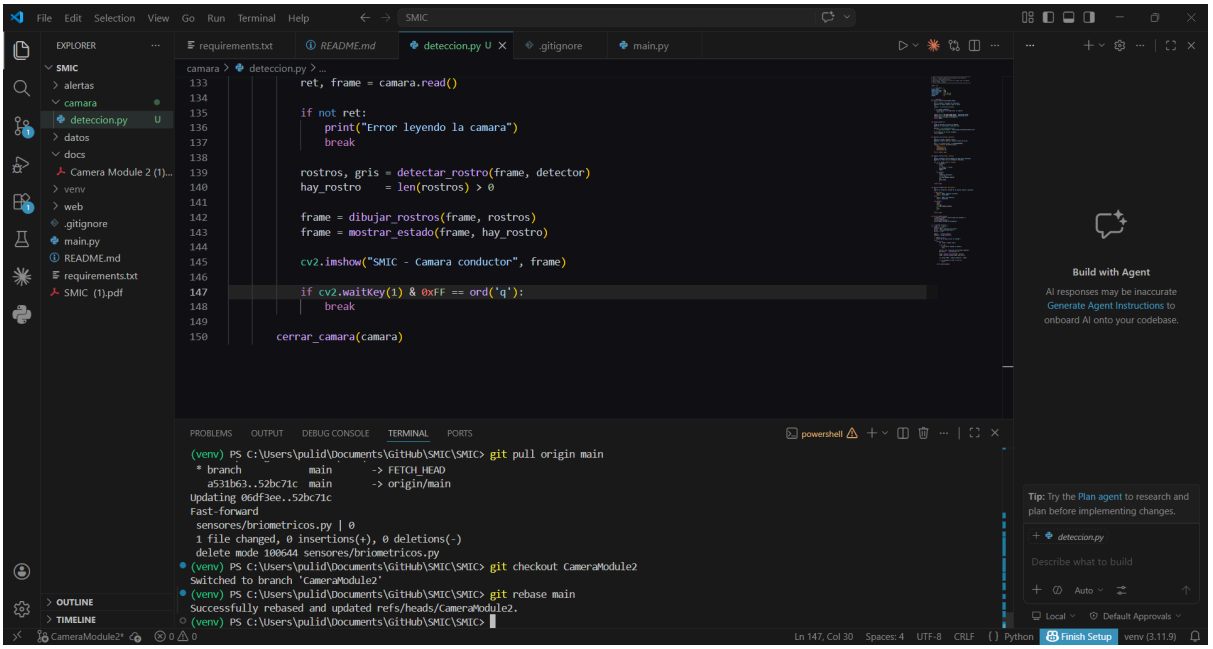
git add .

Hacemos el commit

git commit -m “Eliminar carpeta sensores - no se utilizara en el proyecto”

Subimos a github

git push origin CameraModule2



CODIGO:

Primero importamos la libreria del OpenCV (cv2) esta biblioteca nos permitira abrir la camara, leer los frames y dibujar sobre las imagenes

import cv2

Agregamos constantes :

CAMARA_INDEX = 0

Esta funcion le dice a OpenCV que camara usar, la opcion 0 es la default en la PC y la Raspberry pi 4

RESOLUCION_ANCHO = 640
RESOLUCION_LARGO = 480

Define el ancho y largo de pixeles que utilizaremos

COLOR_VERDE = (0, 255, 0)

COLOR_ROJO = (0, 0, 255)

Los colores en OpenCV se escriben en formato **BGR** (Azul, Verde, Rojo), al revés del RGB habitual. `(0, 255, 0)` significa 0 de azul, 255 de verde y 0 de rojo, o sea verde puro. `(0, 0, 255)` es rojo puro.

GROSOR = 2

El grosor en píxeles de las líneas que dibujamos sobre la imagen.

FUNCIONES:

Definimos la funcion.

Funcion iniciar camara:

def iniciar_camara(index=CAMARA_INDEX):

iniciar_camara(index=CAMARA_INDEX): esta funcion significa que si no le damos ningun numero para buscar una camara, usara la de constante CAMARA_INDEX

Le pedimos a OpenCV que habra la camara numero Index. El resultado va a ser el objeto que represente la camara y guardamos la variable

camara = cv2.VideoCapture(index)

Verificmos si la camara se abrio correctamente con isOpened(), si nos devuelve true esta lista. Si no se abre, imprime el error y devuelve none, y asi nos damos cuenta si algo esta mal

```
if not camara.isOpened():
    print("Error: no se pudo abrir la camara")
    return None
```

Configuramos la resolucion de la camara
CAP_PROP_FRAME_WIDTH Y CAP_PROP_FRAME_HEIGHT son las contantes de OpenCV que representan el ancho y alto

```
camara.set(cv2.CAP_PROP_FRAME_WIDTH,  RESOLUCION_ANCHO)
camara.set(cv2.CAP_PROP_FRAME_HEIGHT, RESOLUCION_ALTO)
```

Imprimimos un mensaje de confirmacion y devuelve el objeto camara para que el resto lo pueda usar

```
print("Camara iniciada correctamente")
    return camara
```

No recibe los parametros porque siempre carga el mismo detector

Funcion cargar detector:

```
def cargar_detector():
```

Carga el detector de rostros de OpenCV. **CascadeClassifier** es un algoritmo que usa archivos XML con patrones de rostro humanos entrenados anteriormente. **cv2.data.haarcascades** es la carpeta donde OpenCV guarda los archivos automaticamente al instalarse. **haarcascade_frontalface_default.xml** contiene el modelo entrenado para detectar rostros de frente

```
detector = cv2.CascadeClassifier(
        cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
    )
```

Confirmamos que cargo y devolvemos el detector para usar

```
print("Detector de rostros cargado")
    return detector
```

Recibimos dos parametros: **frame** es la imagen capturada por la camara en ese instante, y el **detector** es el clasificador que cargamos

Funcion detectar rostro

```
def detectar_rostro(frame, detector):
```

Convierte la imagen de color a escala de grises. El detector de rostros trabaja mejor y mas rapido con imagenes en escala de grises porque contiene menos informacion

```
gris = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

Buscamos los rostros en la imagen gris con **scaleFactor=1.1** , esto significa que reduce la imagen un 10% en cada pasada para detectar rostros de distintos tamaño **minNeighbors=5**

indica cuantas detecciones consecutivas necesita para confirmar que es un rostro real, evita falsos positivos. **minSize=(80, 80)** ignora cualquier deteccion menor a 80x80 pixele, descarta objetos muy pequeños

```
rostros = detector.detectMultiScale(
        gris,
        scaleFactor=1.1,
        minNeighbors=5,
        minSize=(80, 80)
    )
```

Devolvemos la lista de rostros encontrados y la imagen en gris. Cada rostro es una tupla que indica donde esta la imagen

```
return rostros, gris
```

Funcion Dibujar Rostro:

Recibimos el frame original a colory la lista de rostros detectados

```
def dibujar_rostros(frame, rostros):
```

Recorremos cada rostro detectado. Desempaqueta la tupla directamente en cuatro variables: **x** y **y** son las coordenadas de la esquina superior izquierda, **ancho** y **alto** son las dimensiones del retangulo

```
for (x, y, ancho, alto) in rostros:
```

Dibuja el rectangulo sobre el frame. El primer punto **(x, y)** es la esquina superior izquierda y el segundo **(x + ancho, y +alto)** es la esquina inferior derecha. Utilizamos el color verde y grosor definido en las contrastantes

```
cv2.rectangle(
            frame,
            (x, y),
            (x + ancho, y + alto),
            COLOR_VERDE,
            GROSOR
        )
```

Escribe el texto “Conductor detectado” arriba del rectangulo. **(x, y -10)** lo posicion 10 pixeles arriba de la esquina del rectangulo. **cv2.FONT_HERSHEY_SIMPLEX** es el tipo de letra **0.6** esel tamaño de letra

```
cv2.putText(
        frame,
        "Conductor detectado",
        (x, y - 10),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.6,
        COLOR_VERDE,
        2
    )
```

Devuelve el frame con los rectangulos y textos dibujados encima

```
return frame
```

Funcion mostrar estado:

Recibe el frame y un booleanos que indica si hay o no rostro detectado

```
def mostrar_estado(frame, hay_rostro):
```

Elegimos el texto y color segun si hay conductor o no. Indica verde cuando esta presente, rojo cuando no esta.

```
if hay_rostro:
    texto = "SMIC: Conductor presente"
    color = COLOR_VERDE
else:
    texto = "SMIC: Sin conductor"
    color = COLOR_ROJO
```

Escribe el estado en la posicion **(10, 30)**, o sea 10 pixeles desde la izquierda y 30 desde arriba. Siempre visible en la esquina superior izquierda

Funcion cerrar camara:

camara.release() libera el hardware de la camara para que otros programas puedan usarla. **cv2.destroyALLWindows()** cierra todas las ventanas que abrio OpenCV. Sin esto, la camara quedaria bloqueada y habria que reiniciarla la PC para la libreria

```
def cerrar_camara(camara):
    camara.release()
    cv2.destroyAllWindows()
    print("Camara cerrada correctamente")
```

Programa Principal:

Solo se ejecuta si corremos el archivo directamente. Imprime el encabezado en la terminal

```
if __name__ == "__main__":
    print("=" * 45)
    print("  SMIC - Deteccion de rostro")
    print("  Presiona Q para salir")
    print("=" * 45)
```

Llama a las dos funssiones de inicializacion y guarda los resultados en variables

```
camara   = iniciar_camara()
    detector = cargar_detector()
```

Si la camara no puede abrirse, muestra error y no hace nada mas

```
if camara is None:
        print("No se pudo iniciar el sistema")
```

Si la camara esta lista, entra en un bucle infinito. Esto es lo que hace que el video sea continuo, lee frame por frame sin parar

```
else:
        while True:
```

Leemos un frame de la camara. **ret** es **true** si la lectura fue exitosa, **frame** es la imagen capturada en ese instante

```
ret, frame = camara.read()
```

Si falla la lectura, imprime error y salimos del bucle.

```
if not ret:
        print("Error leyendo la camara")
        break
```

Detectan los rostros en el frame actual **len(rostros) > 0** es **true** si encontro al menos un rostro, **false** si no encontro ninguno

```
rostros, gris = detectar_rostro(frame, detector)
        hay_rostro    = len(rostros) > 0
```

dibuja los rectangulos y el estado sobre el frame. Cada funcion recibe el frame, le agrega cosas encima y devuelve el frame modificado

```
frame = dibujar_rostros(frame, rostros)
        frame = mostrar_estado(frame, hay_rostro)
```

Muestra el frame en una ventana con el titulo “SMIC - Camara conductor”. Esta linea se ejecuta 30 veces por segundo aproximadamente, creando el efecto de video en vivo

```
cv2.imshow("SMIC - Camara conductor", frame)
```

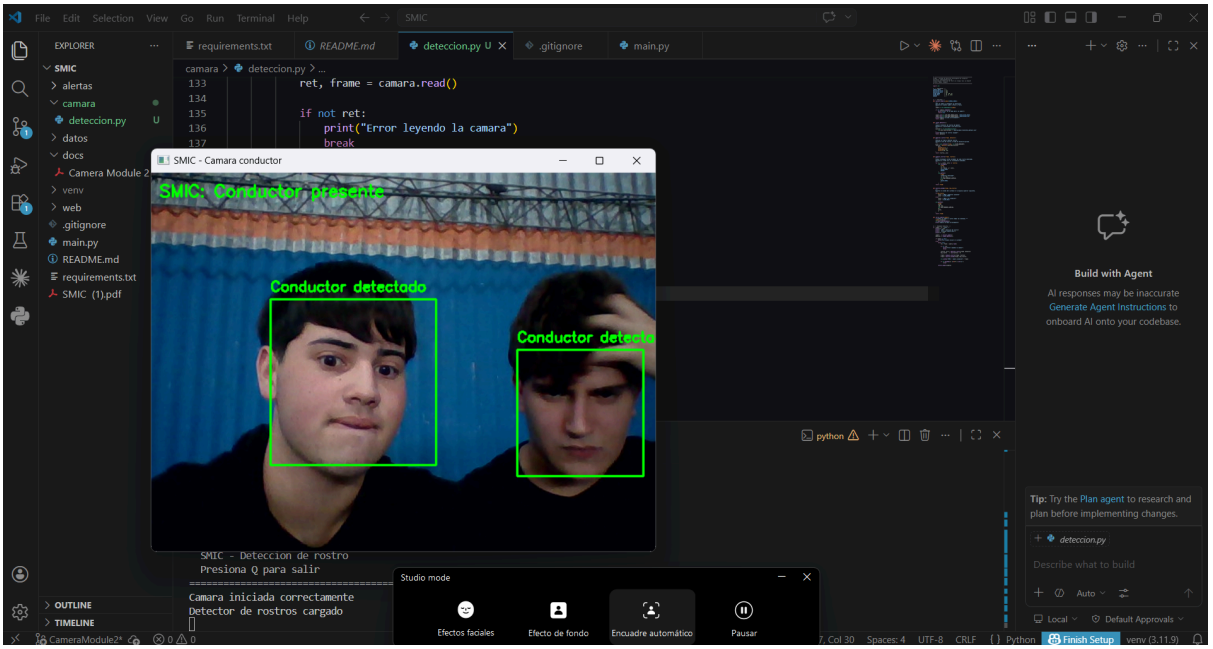
cv2.waitKey(1) espera 1 milisegundo para detectar si se presiono alguna tecla **&0xFF** es una operacion de bits para la compatibilidad entre los sistemas. **ord(‘q’)** es el codigo numerico de la letra Q. Si se presiona Q sale del bucle

```
if cv2.waitKey(1) & 0xFF == ord('q'):
        break
```

Cuando salimos del bucle, libera la camara y cierra la ventana

```
cerrar_camara(camara)
```

EJECUCION:

Ejecutamos el codigo:

python camara/somnolencia.py